

Revshare

[revshare architecture.pdf](#)

REVSHARE PIPELINE

revshare architecture.pdf

REVSHARE PIPELINE

SUMMARY -

Instant_Ink Revshare , Instant_Toner Revshare, Instant_SMB Revshare

****Recent Code Changes:****

****28-01-2025 - Pay as you Print (Opp) Changes:****

- Opp (Pay as you Print) plans added to the same bucket as 10 and 25 page plans
- New columns from source: plan_overage_block_price_cents, billing_cycle_charged_overage_blocks
- Added new column billing_cycle_Opp_block to represent Opp billing plans separately
- Added new global rule to filter out all the 0 enrollment plan subscribers (zero_page_plan_enrollments)
- Master plan price table has additional column for Opp used for calculation
- Created total_price_overage_blocks to represent total price of overage blocks subscribers have used
- Modified final_revshare calculation to include Opp billing_cycle_plan_pages

****26-03-2025 - Modularisation Phase 2:****

- Modularised Retailer/Distributor rules and store details
- Created separate notebooks for each module with functions to be called: retailer_rules.py, Store_details.py, global_rules.py, exceptions.py, master_plan_price.py
- Store details function is same for all notebooks, but merge_plans_bounty and merge_plans_revshare are different for bounty and revshare

****26-03-2025 - Canary Island Changes:****

- retailer_kit_printer_org_details column logic updated to use the pin code of the original retailer instead of the latest retailer
- IDE implemented canary_flag as a boolean true/false field to identify Canary Island subscribers easily

****23-04-2025 - Opp Page Block Column Fix:****

- Fixed billing_cycle_Opp_block column for non-consented data distribution
- Previously, the column was not distributed according to retailer mix, causing duplicacy
- Fixed by distributing the column: billing_cycle_Opp_block = billing_cycle_Opp_block * retailer_mix_round_off

****27-07-2025 - Yearly Plan Changes:****

- Added new global rule (yearly_plan_non_us) to eliminate Yearly Plan records in countries other than US
- For yearly plans, billing_cycle_plan_pages is divided by 12
- Added yearly_retailer_comp_eligibility_ts in places where coalesce(last_charge_attempt_timestamp, billing_cycle_end_time) was used
- Added specific amount calculation for yearly plans: final_revshare/((billing_cycle_plan_pages * 12)/yearly_page_balance_used_in_current_month)
- charge_complete_flag is set to True for yearly plans

****24-09-2025 - Canary Partners Segmentation:****

- Fixed incorrect billing attribution to both Canary and Non-Canary groups
- Updated retailer mix calculation logic by including canary_flag in both country count and retailer count aggregations
- Made canary_flag false for all countries other than ES (Spain)
- Added canary_flag to the group by of retailer count and country count for allocation
- Added canary_flag to the join condition for allocation

Pipeline data flow structure -

1)Start -

2)Source Data –

Data from team_iide_prod.gold_ii.compensation_retailers_monthly_snapshot

Read data from catalog using read_from_catalog function.

Write data to delta tables using upsert/upsert_consolidated functions.

****New columns added to source query:****

- plan_overage_block_price_cents, billing_cycle_charged_overage_blocks (for Opp/Pay as you Print)
- canary_flag, plan_bill_frequency, invoice_items_created_at_ts (for Canary and Yearly plans)
- yearly_page_block_id, yearly_page_balance_used_in_current_month, yearly_payment_successful_ts, yearly_payment_successful_flag, yearly_retailer_comp_eligibility_ts (for Yearly plans)
- page_block_enrollment_flag, plan_overage_block_size (for Opp plans)

3)Define Parameters –

We will pass 6 parameters i.e

InputDict – Used to mention all the countries and retailers.

StartDateList – Used for custom catch up.If not mentioned default is 3months.
 EndDateList - Used for custom catch up.If not mentioned default is last date of the reporting month.
 Program_typeList – InstantInk,Instant Toner ,SMB Toner Mono,SMB Toner colour.
 Reporting_month – The month for which pipeline runs.
 MetaData – Used to mention reason of running pipeline.
 FreezeMix – For yes, last months mix will be taken and no mix will be calculated.For No, new mix will be calculated.
 refresh_type – Options: Audit, dev, Standard, testing. Controls checkpoint data saving behavior.
 feature_name – Feature name for dev testing.
 table_names – Table names for dev testing.
 Required libraries should be imported.
 Adding delta storage paths and add configuration parameters for better performance.

4)Add Custom Calendar for US - office depot and staples –

Table - app_bm_instant_ink_ops.custom_accounting_calendar

Loading country wise vat(Tax) percentage table.
 Creating widgets to get the input-parameters.
 Fetching latest snapshot-table - team_iide_prod.gold_ii.compensation_retailers_monthly_snapshot
 Read Job Parameters for processing input to get startDateList, endDateList in required format

****Modular Imports:****

The following common utility notebooks are imported:
 - common_utils - Common utility functions
 - global_rules - Global rules for eligibility
 - exceptions - Exception handling for specific retailers
 - master_plan_price - Plan price lookup and calculations
 - retailer_rules - Retailer-specific rules
 - Store_details - Store information lookup

5.Modify Start and End Date for custom calendar considerations and Some retailers such as staples, office depot.

****Source Data Transformations:****

After loading source data, the following transformations are applied:
 - plan_overage_block_size: Set to 0 for non "Pay as you Print" plans
 - billing_cycle_plan_pages: Divided by 12 for Yearly plans
 - plan_pages_proxy: Mapping for allocation (0,1015, 2550, 500300, 1500700)
 - price_match_flag: Created for cross-country pricing validation
 - billing_cycle_0pp_block: Created for 0pp plans (equals billing_cycle_charged_overage_blocks when billing_cycle_plan_pages=0)

****Canary Flag Processing:****

- retailer_kit_printer_org_details is updated based on canary_flag for ES (Spain) country
 - If canary_flag is True and country is ES, "_canary" suffix is appended to compensation_retailer_name
 - canary_flag is set to False for all countries other than ES

****Yearly Plan Processing:****

- For yearly plans, relevant_month is calculated using yearly_retailer_comp_eligibility_ts instead of last_charge_attempt_timestamp
 - charge_complete_flag is set to True for yearly plans

6.Query Completer Privacy -Function to generate dynamic query to load the source data based on input details.

7.Defining a functions to create logs (d0,df1..etc) for the delta tables by additional logging detail to existing delta logs .

Upsert Algorithm

Find Matches:Compare new data with what we already have to see if anything matches.
 Update Matches:If we find a match, update the existing information with any changes.
 Insert New Data:If it's completely new, Insert it to our records.
 Handle Differences:
 If data that doesn't match or is missing for the different run time parameters.
 If data is not there in previous version then mark it as irrelevant.
 If it data is not there in current version but present in previous version mark as unmatched.

Apply Changes:Apply the changes based on the conditions.

Uses reporting_month also as an parameter while upserting,This ensures that all dataframes at checkpoints are combined and, we have a consolidated dataframe at each stage,baseDestLoc is changed to '[s3://dataos-core-prod-team-iibi-iipa/data_products/prod/revshare/standard/consolidated/](#)'.Loading source data using constructed query and some tranformations are done as per requirement such as Remove the substring 'Page Plan' from the values in the enrollment_plan column and cast it to integer, making it suitable for joining operations and cast columns to DateType and calculate relevant_month. Here the data is divided into Data and Data pre-privacy. The subscribers who come after july 2022 comes under Data. The subscribers who come before july 2022 comes under Data pre privacy.
 And then on these divided data global rules will be applied next.

5) Global Rules flag created –

Applying global rules on revshare data. Customers who pass these rules will have further rules applied.

Added another condition: comp_retailers of billing_cycle_free_month_type is null for free_months_deferred global rule

Global rules-

Payment_successful_not_suspended when comp_retailers of suspended_entire_cycle_flag is False

Payment_successful_successful_charged when comp_retailers of charge_complete_flag is True

Billingcyclestatus_non_forgiven when comp_retailers of billing_cycle_status_code is not equal to FORGIVEN

Billingcyclestatus_non_prepaid when comp_retailers of billing_cycle_status_code is not equal to PEGASUS-SUCCESS

Flip_enrollment_all when comp_retailers of flip_type is equal Flip or comp_retailers of flip_type is equal Hybrid Flip & comp_retailers of flip_compensation_eligible_flag is True

Legacy_rule_kitless_with_promocode when comp_retailers of kitless_revshare_eligibility_legacy is True & comp_retailers of flip_type is not equal to Flip & comp_retailers of flip_type not equal to Hybrid Flip

Kitless_21_days_non_legacy when comp_retailers of kitless_revshare_eligibility_non_legacy is True & comp_retailers of flip_type not equal to Flip & comp_retailers of flip_type not equal to Hybrid Flip

Kit_all_eligible when comp_retailers of enrollment_type is not equal KITLESS & comp_retailers of flip_type is not equal Flip & comp_retailers of flip_type is not equal to Hybrid Flip

Free_months_deferred when comp_retailers of billing_cycle_free_month_type is Null or comp_retailers of billing_cycle_free_month_type is equal to full or comp_retailers of Billing_cycle_free_month_type is in RemainingCustomerSatMonth, RemainingPrinterReplacementMonth, RemainingReferAFriendMonth.

Lups_no_revshare when, inverse of enrolled_on_date < 2020-11-01 & enrollment_plan is equal to 15 & billing_cycle_plan_pages is equal to 15.

****Zero_page_plan_enrollments**** (NEW - 28-01-2025) when page_block_enrollment_flag is True - filters Opp/Pay as you Print enrollments

****Yearly_plan_non_us**** (NEW - 27-07-2025) when plan_bill_frequency is 'Yearly' AND country_id_coalesced is NOT 'US' - eliminates yearly plan records for non-US countries

6) Exceptions –

DE-MSH

GB-Dixons

AU/NZ partners

exceptions_list

Exception: MSH gets a share of the revenue from P2 kitless customers, for those who enrolled between June 2016 and August 2018. This is an exception to the usual rule.

The code iterates over each DataFrame in compRetailerList and adds a new column 'kitless_21_days_non_legacy'.

We will make the revshare payment to DE-MSH, when the following conditions are met True:

The 'enrolled_on_date' is between '2016-06-01' and '2018-08-01'.

The 'compensation_retailer_name' is 'MSH'.

The 'country_id_coalesced' is 'DE'.

The existing value of 'kitless_21_days_non_legacy' column is False.

Otherwise, the value of 'kitless_21_days_non_legacy' remains the same as before.

The code updates the kitless_21_days_non_legacy column for the entries in the comp_retailers DataFrame that meet the following criteria:

They were enrolled between June 1, 2016, and August 1, 2018.

The compensation_retailer_name is 'MSH'.

The country_id_coalesced is 'DE'.

The kitless_21_days_non_legacy column is currently False.

If all these conditions are met, the column value is set to True; otherwise, it remains as it was.

get all the exceptions

query = "select * from app_bm_instant_ink_ops.exceptions_list where exception_type != 'Bounty and exceptions_df.count()==0):

print('No exceptions applicable

This code processes a DataFrame to fill in missing exception end dates and apply specific exceptions to an 'eligibility_exception' column based on a series of conditions.

Each exception is applied one by one, updating the column if the conditions specified by the exception are met.

Applying exception on revshare data

Logic APJ GB Dixons

Create No Free Month Column For Dixons GB

16-02-2024 updated retailer_compensation.cross_country_compensation, bi_fact.compensation_retailers_monthly_snapshots, and cr.snapshot_date = {max_snp_date}

GB DIXONS Exception: If the customer enrolled before September 2020 or has more than 4 free months, will pay revshare for the retailer

Select distinct combinations of billing_cycle_id and Calculate the total number of free months for each subscription_id. Use a window function to compute cumulative counts
For each row, if billing_cycle_free_month_type is null or empty, count it as a free month assign 1, otherwise, count it as a non-free month assign 0
If the current month's billing_cycle_free_month_type is NULL too, add 1 to the total_no_free.
Check whether free months have exhausted and mark them eligible accordingly.

AU and NZ Exception:

Create Free Months Column For AU and NZ

AU-NZ Exception: RevShare for all AU/NZ partners starts soon after the first 2 trial months are over.

19-02-2024 updated retailer_compensation.cross_country_compensation, bi_fact.compensation_retailers_monthly_snapshots, and cr.snapshot_date = {max_snp_date}

When count>2, override "free months deferred" global to true then check whether 2 free months have exhausted and mark them eligible accordingly.

Replace free_months_deferred = False to free_months_deferred = True in eligibility_criteria

Track as compensated, even if in free months. i.e. If free_months_deferred rule has been overridden due to this exception. Add a new column for the same.

Applying exceptions:

comp_retailers_pre_privacy = dixonException(compRetailerList[1], '2022-06-01')

comp_retailers_pre_privacy = au_nzException(comp_retailers_pre_privacy, '2022-06-01')

7) Global_rules_exceptions_combined_is_created –

Create global_rules_exceptions_result column and mark it pass if that row satisfies all program specific global rules conditions and expetions.

If program_typeList is equal to Instant Ink print II if not print IT.

Filter the data with the highest 'distributor_zyme' value within each 'billing_cycle_id' partition. The code ensures that there are no duplicate billing_cycle_id entries.

8) Check point – 0

This end check point 0, all these data come into df0.

Source data added.

Exceptions are added.

Global rules flag added.

8) Retailer Rules columns are added –

Tables -

mapping_rules_index,
mapping_retailer_compensation,
mapping_retailer_compensation_rule

Get only the approved rules for the current reporting_month and get the complete retailer_compensation_rules i.e from retailer_rules = load_retailer_rules_complete()

Join with index table to get retailer rules only for the reporting_month i.e retailer_rules = index.join(retailer_rules, index.retailer_rules_index == [retailer_rules.id](#))

Then load_retailer_rules and few transformations are done as per business requirements.

Adjustments To ensure unconsented logic applies to details and source columns as well

Privacy and pre -privacy data is sorted and cleaned as required.

Phase 2 Allocation Process Algorithm

Read current months data : comp_retailers = readDataFromSpectrum(query_all).

Read pre-July data : comp_retailers_pre_privacy = readDataFromSpectrum(query_pre_privacy).

Create cohorts:

Consented Enrollments: comp_retailers_consented.

Non-Consented Enrollments Pre-July 22: comp_retailers_non_consented_pre_jul_22.

Non-Consented Enrollments Post-July 22: comp_retailers_non_consented_post_jul_22.

Perform Pre-July Allocation: Join comp_retailers_non_consented_pre_jul_22_passed_global with mix_combined_pre_jul to obtain comp_retailers_non_consented_pre_jul_allocated_passed_global.

Perform Post-July Allocation: Join comp_retailers_non_consented_post_jul_22_passed_global with mix_combined_post_jul to obtain

comp_retailers_non_consented_post_jul_allocated_passed_global.

Final Allocation: Combine (Union) comp_retailers_consented_passed_global, comp_retailers_non_consented_pre_jul_allocated_passed_global, and comp_retailers_non_consented_post_jul_allocated_passed_global to obtain the final allocated dataset.

Filter for consented i.e comp_retailers_consented = comp_retailers.filter of compensation_retailer_name != 'Not Attributed'

Filter for non-consented i.e comp_retailers_non_consented = comp_retailers.filter of compensation_retailer_name= 'Not Attributed'

Filter for Passed Global Rules.

8)Retailer_mix calculations Pre-july –

Calculate Mix for Pre-July 22 subscriber using comp_retailers_pre_privacy_passed_global, resulting in mix_combined_pre_jul.

Mix for Pre July 22 Subscribers :

Calculate subscriber count in each country - country- subscriber_count and calculate subscriber count of retailers for each country - retailer_subscriber_count_in_county

Calculate retailer mix by dividing country- subscriber_count with retailer_subscriber_count_in_county

****Group By columns for country count:**** country_id_coalesced, ****canary_flag**** (NEW - 24-09-2025), relevant_month, enroll_fy, plan_pages_proxy, program_type

****Group By columns for retailer count:**** country_id_coalesced, ****canary_flag**** (NEW - 24-09-2025), relevant_month, compensation_retailer_name, enroll_fy, plan_pages_proxy, program_type, retailer_kit_printer_org_details_mod, printer_retailer_source_mod

Logic: If(freezeMix == 'No')

if mix is no,then calculate the mix.If mix is yes, then take the mix from previous run.

9)Retailer_mix calculations Post-july –

Calculate Mix for Post-July 22 subscriber using comp_retailers_consented_post_jul_22_passed_global, resulting in mix_combined_post_jul.

Mix for Post July 22 subscribers :

Calculate subscriber count in each country - country- subscriber_count and calculate subscriber count of retailers for each country - retailer_subscriber_count_in_county

Calculate retailer mix by deviding country- subscriber_count with retailer_subscriber_count_in_county

****Group By columns for country count:**** country_id_coalesced, ****canary_flag**** (NEW - 24-09-2025), relevant_month, enroll_fy, plan_pages_proxy, program_type, reporting_month

****Group By columns for retailer count:**** country_id_coalesced, ****canary_flag**** (NEW - 24-09-2025), relevant_month, compensation_retailer_name, enroll_fy, plan_pages_proxy, program_type, reporting_month, retailer_kit_printer_org_details_mod, printer_retailer_source_mod

Logic: if(freezeMix == 'No'):

if mix is no,then calculate the mix.If mix is yes, then take the mix from previous run.

10)Final Allocation-

Pre july allocation-comp_retailers_non_consented_pre_jul_22_passed_global.is joined with mix_combined_pre_jul on country_id_coalesced, ****canary_flag**** (NEW - 24-09-2025), enroll_fy, plan_pages_proxy, program_type,reporting_month

Post july allocation-comp_retailers_non_consented_post_jul_22_passed_global is joined with mix_combined_post_jul on country_id_coalesced, ****canary_flag**** (NEW - 24-09-2025), relevant_month, enroll_fy, plan_pages_proxy, program_type, reporting_month

Final allocation- final_eligible_allocated using comp_retailers_consented_passed_global and

comp_retailers_non_consented_pre_jul_allocated_passed_global is joined by comp_retailers_non_consented_post_jul_allocated_passed_global

Now filter for only country-retailer combinations decided and create Temp View first

query_filter_relevant_eligible_allocated using final_eligible_allocated_sql

Then data cleaning is done and stored in -stored in comp_retailers_sql

****NEW (23-04-2025) - Opp Block Distribution Fix:****

billing_cycle_Opp_block is multiplied by retailer_mix_round_off to ensure proper distribution for non-consented data:

billing_cycle_Opp_block = billing_cycle_Opp_block * retailer_mix_round_off

11)Check point -1

This end check point 1, all these data come into df1.

Global rules pass.

Allocation is done.

****billing_cycle_Opp_block distributed according to retailer mix (NEW - 23-04-2025)****

12)Join retailer rules with result of allocated data –

1.Joining data post allocation with final retailer rules data on country , allocated_retailer,allocated_details,allocated_source,reporting_plan to get data from rules data table, rule_date_eligibility is generated by checking if enrolled on date is between rule start date and rule end date.

13)Create Rule Date Eligibility –

Checks if rule dates fall between last_charge_attempt/billing_cycle_end_time of subscription_id

****NEW (27-07-2025) - Yearly Plan Rule Date Eligibility:****

For yearly plans (plan_bill_frequency = 'Yearly'), rule date eligibility uses yearly_retailer_comp_eligibility_ts instead of coalesce (last_charge_attempt_timestamp, billing_cycle_end_time)

****NEW - Billing Plan Eligibility Flag:****

A billing_plan_eligibility flag is added to identify if the billing cycle plan pages match with retailer rules (summary mismatch solution)

14) Plan price implementation –

Instant Ink Source Pricing Implementation

Load Data from Source (CR): Include the plan_price_in_cents column while loading data from the source (CR).

Load Master Plan Pricing Data: Load the master plan pricing table created using the dim plan table bi_dim.iink_plan.

****NEW (28-01-2025) - Master Plan Price for Opp:****

Master plan price table includes additional column planprice_Opp for Opp (Pay as you Print) plans

****NEW (27-07-2025) - Yearly Plan Price:****

For yearly plans, plan_pages_number is divided by 12 in the master plan price join

Join Pricing Data to Handle Cross-Country Pricing: Join the master plan pricing data with the loaded CR data using the following conditions:

Match country codes, match billing cycle plan pages, match program type, ****match plan_overage_block_size (NEW)****, ****match plan_bill_frequency (NEW)**** and ensure the join occurs within the valid date range.

If cross-country pricing is applicable: Check if the country IDs differ.

If so, replace plan_price_in_cents with plan_price_amount from the master plan price table.

Join with Tax Rate Data (VAT Percentage):

Join the overall data with the country tax rate data using join_country as the key.

Create the join_country column based on allocated_details because Tax table has different country codes for Spain(ES) and Spain canary(CI): If allocated_details contains "CANARY", set join_country to "CI". Its not applicable here for distributor.

Otherwise, set join_country to country_id_coalesced.

Adjust Plan Price Considering VAT: Calculate the planprice by adjusting plan_price_in_cents based on the VAT percentage.

Handle Null Plan Prices: If both planprice and plan_price_in_cents are null, set planprice to 0, otherwise, keep planprice as it is.

Mathematical Formula: $\text{planprice} = \text{round}(100 \times \text{plan_price_in_cents} / (100 + \text{vat_percentage}), 0)$

15) Check point -2

This ends check point 2, all these data come into df2.

Retailer rules added.

Rule date eligibility flag is added.

Plan price columns are added.

****Billing plan eligibility flag is added (NEW)****

16) Rule Date Eligibility is true ? –

1. Filtering cases only where rule data eligibility is true.

17) Remove duplicate rules and take latest rule –

Removing duplicates and We filter the most recent rule.

18) Create compensation_type – 1. Now we create Compensation type and categorize data if its Kit revshare, Kitless revshare or Not Applicable. It is decided by the following conditions:

Kit revshare: When enrollment_type is not kitless and enrollment date falls within kit contract dates.

Kitless revshare: When enrollment_type is kitless and enrollment date falls within kitless contract dates. If the enrollment date is before November 1, 2016, and falls within the Kit contract dates, the compensation type is Kitless Revshare.

Not Applicable: If it doesn't fall under both of the cases above.

2. We create json rules criteria is created as pass.

19) Create contract_eligibility_criteria-

Table -overall_joined_ruledate_filtered

1. Contract_eligibility_criteria is created as 'Pass' for all compensation_type except 'Not Applicable', for which it is fail in the table 'overall_joined_ruledate_filtered'.

20) Identify legacy customers –

Subscribers who enrolled before November 1, 2016

21) Create revshare_eligibility –

1. Here we decide the revshare eligibility if it is given, not given or eligible but no revshare:

Eligible but no revshare: When compensation type is Kit revshare and kit revshare is 0 or when compensation type is kitless revshare and kitless revshare is 0 and json rules criteria is pass and contract eligibility criteria is pass.

Revshare given: json rules criteria is pass and contract eligibility criteria is also pass.

No Revshare: When both the above conditions are not satisfied.

22) Get frequency of reports-

Retailer_compensation.mapping_rules_index

1. Join with mapping_rules_index to get frequency related information.

23) Check point -3

This end check point 3, all these data come into df3.

Retailer rule date eligibility is true.

Revshare eligibility flag is applied.

Contract eligibility criteria is applied.

24) Revshare_Eligibility is Revshare given? –

Filtering for only revshare_eligibility i.e. Revshare given.

25) Store details added–

1. For store details, get store details from "app_instant_ink_bi_fact.stores_per_subid"

2. We transform these columns oobe_timestamp, cancellation_date into date format. We join the store table with result on subscription_id. We create 4 columns which are store_id, store_key, store display name, store comments text.

3. To create store_id these are the conditions:

If the value of the column 'enrollment_type' is 'Prepaid_Standalone', and the 'ori_card_store_id' is null while the 'ori_printer_store_id' is not null, then the value of the 'store_key' column is set as the value of the 'ori_printer_store_key' column.

Otherwise, if the value of the column 'enrollment_type' is not equal to 'Kitless', then the value of the 'store_key' column is set as the value of the 'ori_card_store_key' column.

If none of the above conditions match, the value of the 'store_key' column is set as the value of the 'ori_printer_store_key' column.

4. Create revshare rate column which has 2 values kit revshare or kitless revshare. And it depends on compensation type column.

5. We add mapped store logic for MSH. Mapped store will be true only for these conditions:

retailer_kit_printer should be equal to MSH.

country_id should be AT and DE.

When country_id = AT and store_comments_text starts with A and store_comments_text is not null and store display name is not null.

When country_id = DE and store_comments_text starts with M OR S and store_comments_text is not null and store display name is not null.

26) Decimal round off columns created–

Retailer_mix_round_off and Amount columns are created.

1. We create retailer mix round off column by rounding off retailer mix into 2 decimal places.

****NEW (28-01-2025) - Opp (Pay as you Print) Calculations:****

- total_price_overage_blocks: Created when billing_cycle_plan_pages = 0 and billing_cycle_charged_overage_blocks is not null

- Formula: $\text{billing_cycle_charged_overage_blocks} * \text{planprice_Opp}$

- final_revshare for Opp: $\text{round}((\text{revshare_rate} * \text{planprice_Opp}) / 10000, 2)$

- For non-Opp plans: $\text{final_revshare} = \text{round}((\text{revshare_rate} * \text{planprice}) / 10000, 2)$

****Amount column calculation (Updated for Opp and Yearly plans):****

For US country:

- When billing_cycle_plan_pages = 0 (Opp): $\text{round}((\text{revshare_rate} * \text{total_price_overage_blocks}) / 10000, 2)$

- When billing_cycle_plan_pages != 0 and plan_bill_frequency != 'Yearly': final_revshare

- ****NEW (27-07-2025)**** When billing_cycle_plan_pages != 0 and plan_bill_frequency = 'Yearly': $\text{final_revshare} / ((\text{billing_cycle_plan_pages} * 12) / \text{yearly_page_balance_used_in_current_month})$

For non-US countries:

- When billing_cycle_plan_pages = 0 (Opp): $\text{round}(((\text{revshare_rate} * \text{total_price_overage_blocks}) / 10000) * \text{retailer_mix_round_off}, 2)$

- Otherwise: $\text{round}(\text{final_revshare} * \text{retailer_mix_round_off}, 2)$

27)USD Conversion–

Table - team_iide_prod.gold_css_dim.dim_currency_rate (Updated from App_BM_Instant_Ink_Ops.base_hist_curr_val_v2)

1.We define dollar conversion table:

Table : team_iide_prod.gold_css_dim.dim_currency_rate

Logic : We join dollar conversion table to convert the local currency amount to USD. The table is pivoted by currency_code to get conversion rates for each currency.

28)Grossup for US–

Table – team_iide_prod.gold_ii_pii.printer_consents_country_agg_cr (Updated from bi_fact.printer_consents_country_agg_CR)

1.We define the gross up table for revshare,

Table : team_iide_prod.gold_ii_pii.printer_consents_country_agg_cr

We take the latest gross up value by partitioning according to country_id, program type,reporting_month,order by created_at.

Logic : We create gross up column from amount by dividing the amount with telimetry_optin_and_blank_percent and multiply with 100.

This is done because US comes under phase one logic. We gross up the amount with the percentage present in this table which is updated every month.

2.1st we convert all amount to USD then gross up only for US and convert the amount gross up column to USD.

29)Check point -4

This end check point 4, all these data come into df6.

Final Revshare given subscribers.

30)End